

Programas, Processos e Threads em um PC

Relatório Técnico — Módulo 1
Programação Paralela e Concorrente

Lucas Lustosa Dias — Matrícula: 2410512
Tiago Taumaturgo — Matrícula: 2410518
Curso de Ciências da Computação
26 de agosto de 2026

1 RESUMO

Este relatório apresenta os resultados das atividades práticas do Módulo 1, cujo objetivo é diferenciar programa, processo e thread por meio de observação direta do sistema operacional. Foram realizadas quatro atividades utilizando scripts Python com a biblioteca `psutil` para monitoramento de processos em ambiente Windows. Os experimentos comprovaram que: (i) múltiplas instâncias de um programa geram processos independentes com PIDs distintos; (ii) um único processo pode conter múltiplas threads; (iii) processos formam hierarquias pai-filho; e (iv) processos em loop consomem 100% de um núcleo de CPU. Os resultados experimentais confirmaram a fundamentação teórica estudada em sala de aula.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Programa vs. Processo

Um **programa** é um arquivo executável estático armazenado em disco, contendo instruções de código de máquina. Um **processo** é uma instância em execução de um programa, carregado na memória pelo sistema operacional, com recursos alocados (memória, descritores de arquivo, etc.) [1].

Cada processo é identificado unicamente por um **PID** (Process IDentifier), um número inteiro atribuído pelo kernel. O **PPID** (Parent Process IDentifier) identifica o processo que criou (spawned) o processo atual, formando uma hierarquia em árvore [2].

2.2 Processos e Threads

Uma **thread** (linha de execução) é a menor unidade de processamento que pode ser agendada pelo sistema operacional. Um processo pode conter uma ou mais threads que compartilham o mesmo espaço de endereçamento de memória, mas possuem seus próprios registradores e pilha [1].

A relação entre processos e threads pode ser expressa como:

$$P = \{T_1, T_2, \dots, T_n\} \quad \text{onde } n \geq 1 \quad (1)$$

onde P é um processo e T_i são suas threads. No Windows, o número de threads é indicado pela propriedade `num_threads` [3].

2.3 Hierarquia de Processos

No Windows, processos formam uma hierarquia onde cada processo (exceto o processo raiz do sistema) tem um processo pai. A cadeia de ancestrais pode ser rastreada recursivamente através do PPID:

$$\text{Cadeia}(p) = p \rightarrow \text{parent}(p) \rightarrow \dots \rightarrow p_{\text{raiz}} \quad (2)$$

2.4 Consumo de CPU

O percentual de CPU de um processo é calculado pela razão entre o tempo de CPU consumido e o tempo real decorrido:

$$\text{CPU}_{\%} = \frac{t_{\text{cpu}}}{t_{\text{real}}} \times 100\% \quad (3)$$

Um processo em loop infinito consumirá aproximadamente 100% de um núcleo lógico de CPU [2].

3 PROCEDIMENTO EXPERIMENTAL

3.1 Ambiente de Execução

Os experimentos foram realizados no seguinte ambiente:

- **SO:** Windows 11 Home (build 26200)
- **Python:** 3.11.9
- **Biblioteca:** psutil 6.1.0
- **Processador:** Intel Core i5-1335U (10 núcleos físicos, 12 lógicos)
- **Ferramentas nativas:** Gerenciador de Tarefas e PowerShell

Cada atividade foi executada **duas vezes**: primeiro com a ferramenta nativa do Windows e depois com o script Python equivalente, de modo a permitir a comparação entre as duas fontes de informação (Questão 4).

3.2 Atividade 1.1 — Programa vs. Processo

Objetivo: Comprovar que múltiplas instâncias do mesmo programa criam processos independentes.

Procedimento:

1. Abrir o Bloco de Notas (`notepad.exe`) três vezes manualmente.
2. *Ferramenta nativa:* abrir o Gerenciador de Tarefas (Ctrl+Shift+Esc), acessar a aba **Detalhes** e localizar as três entradas de `notepad.exe` na coluna PID.
3. Anotar os PIDs de cada janela.

4. *Script Python:* executar `listar_processos.py` e conferir se os PIDs coincidem com os do Gerenciador.
5. Fechar uma janela e repetir **as duas** listagens, verificando o desaparecimento do PID correspondente.

3.3 Atividade 1.2 — Threads de um Processo

Objetivo: Verificar que processos contêm múltiplas threads.

Procedimento:

1. *Ferramenta nativa:* consultar o número de threads pelo PowerShell, com `Get-Process -Id <PID> | Select-Object -ExpandProperty Threads`, que lista cada thread com seu identificador individual.
2. *Script Python:* executar `threads_do_processo.py` para analisar o próprio processo Python.
3. Repetir as duas consultas para outro processo (Google Chrome) e anotar o número de threads.
4. Comparar os números obtidos pelas duas ferramentas e entre processos simples e complexos.

3.4 Atividade 1.3 — Árvore de Processos

Objetivo: Visualizar hierarquia pai-filho de processos.

Procedimento:

1. *Ferramenta nativa:* listar processos e seus pais pelo PowerShell, com `Get-CimInstance Win32_Process | Select-Object ProcessId, ParentProcessId, Name`, e subir manualmente a cadeia de PPIDs.
2. *Script Python:* executar `arvore_processos.py`, que percorre a mesma cadeia automaticamente.
3. Confrontar a cadeia obtida manualmente com a reconstruída pelo script.
4. Identificar o processo raiz e desenhar o diagrama da hierarquia.

3.5 Atividade 1.4 — Monitoramento de CPU

Objetivo: Observar consumo intensivo de CPU.

Procedimento:

1. Terminal 1: executar `gerador_carga.py` e anotar o PID exibido.
2. *Ferramenta nativa:* acompanhar o processo no Gerenciador de Tarefas (aba **Detalhes**, coluna CPU, e aba **Desempenho** para a visão gráfica).
3. *Script Python:* Terminal 2: executar `monitor_cpu.py <PID>` simultaneamente ao monitor nativo.
4. Observar o uso de CPU por 30 segundos nas duas ferramentas.
5. Encerrar o processo com Ctrl+C e verificar a queda de consumo em ambas.

4 RESULTADOS E DISCUSSÃO

4.1 Atividade 1.1 — Múltiplas Instâncias

A execução do script `listar_processos.py` retornou três processos distintos do `notepad.exe`, conforme Tabela 1.

Tabela 1: PIDs observados para `notepad.exe`

Janela	PID
Janela 1	8452
Janela 2	9128
Janela 3	9864

Os mesmos três PIDs já haviam sido lidos na aba Detalhes do Gerenciador de Tarefas, sem divergência entre as duas fontes. Ao fechar a Janela 2, o PID 9128 desapareceu de **ambas** as listagens, confirmando que cada janela corresponde a um processo independente. Todos os processos compartilham o mesmo executável em disco (`C:\Windows\System32\notepad.exe`), mas possuem espaços de memória isolados.

Este resultado comprova a fundamentação teórica: um programa (arquivo estático) pode ter múltiplas instâncias em execução (processos), cada uma com seu próprio PID e recursos alocados.

4.2 Atividade 1.2 — Análise de Threads

A Tabela 2 apresenta o número de threads observado em diferentes processos.

Tabela 2: Número de threads por processo

Processo	Threads
<code>python.exe</code> (script)	1
<code>notepad.exe</code>	3
<code>chrome.exe</code> (navegador)	38

A contagem obtida pelo PowerShell (`Get-Process ... Threads`) reproduziu exatamente os mesmos valores, inclusive os identificadores individuais das threads. O script Python simples executou com apenas 1 thread (thread principal), enquanto o navegador Google Chrome apresentou 38 threads. Este resultado era esperado: aplicações modernas utilizam múltiplas threads para paralelizar tarefas como renderização, network I/O, JavaScript execution, etc.

A saída detalhada do script para o processo Python foi:

```
1 Processo PID=12044 - Nome: python.exe
2 Numero de threads: 1
3 Thread ID=12048 tempo_usuario=0.0156s
4 tempo_sistema=0.0312s
```

Listing 1: Saída de `threads_do_processo.py`

Este resultado foi surpreendente inicialmente, pois esperava-se que o interpretador Python utilizasse mais threads. Porém, devido ao GIL (Global Interpreter Lock), o Python executa código em uma única thread por padrão, criando threads adicionais apenas quando explicitamente solicitado via módulo `threading`.

4.3 Atividade 1.3 — Hierarquia de Processos

A execução do script `arvore_processos.py` retornou a seguinte cadeia de ancestrais:

```
1 explorer.exe (PID=4328) ->
2 cmd.exe (PID=10552) ->
3 python.exe (PID=11204)
```

Listing 2: Cadeia de processos

A hierarquia pode ser representada pelo diagrama:

```
explorer.exe (gerenciador)
|
+-- cmd.exe (terminal)
    |
    +-- python.exe (script)
```

A mesma cadeia foi reconstruída manualmente a partir da saída do `Get-CimInstance Win32_Process`, seguindo a coluna `ParentProcessId` de um processo ao outro: os pares PID/PPID coincidiram com os do script em todos os níveis.

O processo raiz visível é o `explorer.exe` (PID 4328), que no Windows atua como o shell gráfico do usuário. Este processo criou o `cmd.exe` quando o Prompt de Comando foi aberto, que por sua vez criou o processo `python.exe` ao executar o script.

No Windows, diferentemente do Linux (onde o processo raiz é sempre `systemd` ou `init` com PID 1), não há um único processo raiz universal. O `explorer.exe` é o ancestral comum para processos iniciados pelo usuário via interface gráfica.

4.4 Atividade 1.4 — Consumo de CPU

O gerador de carga (`gerador_carga.py`) executou em um loop infinito incrementando uma variável. O monitor registrou os seguintes valores de CPU:

Tabela 3: Uso de CPU ao longo do tempo

Amostra	CPU (%)
1	98.7
2	99.2
3	100.0
4	99.5
5	100.0
6	99.8
7	100.0
8	99.3
9	100.0
10	99.6
Média	99.5

A Figura 1 ilustra o comportamento observado visualmente no Gerenciador de Tarefas do Windows.

```

CPU: |#####| 100%
      0%           50%           100%

Antes: |##           | 12%
Durante: |#####| 100%
Depois: |##           | 11%

```

Figura 1: Consumo de CPU antes, durante e após execução

O processo consumiu consistentemente $\sim 100\%$ de **um** núcleo lógico de CPU (equivalente a $\sim 8.3\%$ do total em um processador com 12 núcleos lógicos). O Gerenciador de Tarefas exibiu justamente esse percentual agregado na coluna CPU da aba Detalhes, enquanto o `monitor_cpu.py` reportava $\sim 100\%$: as duas leituras são coerentes, pois `psutil` normaliza o consumo por núcleo e o Gerenciador o expressa em relação ao total da máquina. Após encerrar o processo com Ctrl+C, o consumo voltou imediatamente ao nível basal ($\sim 11\%$) nas duas ferramentas.

Este resultado confirma a Equação (3): um processo em loop infinito sem operações bloqueantes (I/O, sleep) consome 100% do tempo de CPU disponível em um núcleo. O sistema operacional agendou o processo exclusivamente em um único núcleo devido à natureza single-threaded do código Python.

4.5 Comparação entre ferramenta nativa e psutil

A Tabela 4 consolida, para as quatro atividades, o que foi observado com a ferramenta nativa do Windows e com o script Python.

Tabela 4: Ferramenta nativa vs. `psutil` em cada atividade

Ativ.	Ferramenta nativa	Python (<code>psutil</code>)	Confere?
1.1	Ger. de Tarefas: 8452, 9128, 9864	8452, 9128, 9864	Sim
1.2	PowerShell: 1 / 3 / 38 threads	1 / 3 / 38 threads	Sim
1.3	<code>Win32_Process</code> : 4328–10552–11204	4328–10552–11204	Sim
1.4	Ger. de Tarefas: $\sim 8\%$ do total	$\sim 100\%$ de um núcleo	Sim*

* Mesma medida expressa em bases diferentes: o Gerenciador de Tarefas normaliza pelo total de núcleos lógicos da máquina (12), enquanto o `cpu_percent()` do `psutil` usa como referência um único núcleo. $100\% \div 12 \approx 8.3\%$, ou seja, os valores são equivalentes.

Essa foi a **única** diferença encontrada entre as duas abordagens em todo o módulo, e ela é de unidade de medida, não de conteúdo. Nas Atividades 1.1, 1.2 e 1.3 os números foram idênticos, dígito a dígito.

5 CONCLUSÃO — RESPOSTAS ÀS QUESTÕES DO MÓDULO 1

As atividades práticas permitiram validar experimentalmente os conceitos teóricos de programas, processos e threads. Esta seção responde, uma a uma, às **seis questões propostas no roteiro do Módulo 1**. Cada subseção reproduz o enunciado da questão em itálico e apresenta em seguida a resposta fundamentada nos resultados da Seção 4.

5.1 Questão 1 — PID vs. PPID

Qual a diferença entre PID e PPID? Ilustre com um exemplo observado na Atividade 1.3.

Resposta: O PID é o identificador único do próprio processo, atribuído pelo kernel no momento da criação; o PPID é o identificador do processo **pai**, ou seja, daquele que o criou. O primeiro responde “quem sou eu”, o segundo responde “quem me criou”.

O exemplo observado na Atividade 1.3 é a cadeia `explorer.exe` (PID 4328) $\rightarrow$ `cmd.exe` (PID 10552) $\rightarrow$ `python.exe` (PID 11204). O `python.exe` tem PID 11204 e PPID 10552, valor que corresponde exatamente ao PID do `cmd.exe` de onde o script foi lançado. É essa correspondência entre o PPID de um processo e o PID de outro que encadeia a hierarquia em árvore do sistema.

5.2 Questão 2 — Múltiplas instâncias e compartilhamento de memória

Ao abrir três instâncias do mesmo programa, quantos PIDs você observou? Eles compartilham memória entre si?

Resposta: Foram observados **três PIDs distintos** — 8452, 9128 e 9864 (Tabela 1) —, um para cada janela do Bloco de Notas, embora as três executem o mesmo arquivo em disco (`C:\Windows\System32\notepad.exe`).

Não, esses processos não compartilham memória entre si. Cada um recebeu do sistema operacional seu próprio espaço de endereçamento virtual, isolado dos demais: o texto digitado em uma janela não aparece nas outras, e uma falha em uma instância não derruba as demais. Esse isolamento é justamente a garantia de proteção que caracteriza o processo como unidade.

5.3 Questão 3 — Número de threads observado

Quantas threads o processo analisado na Atividade 1.2 possuía? Isso foi surpreendente?

Resposta: O processo Python analisado possuía **apenas 1 thread** (Tabela 2), a thread principal, com o identificador 12048.

Sim, o resultado foi surpreendente em um primeiro momento: esperava-se que o interpretador Python mantivesse threads auxiliares próprias. A investigação posterior mostrou a razão: por causa do GIL (*Global Interpreter Lock*), o CPython executa *bytecode* em uma única thread por vez e só cria threads adicionais quando o programa as solicita explicitamente pelo módulo `threading`. O contraste com o Google Chrome, que apresentou 38 threads no mesmo instante, evidencia como a contagem depende inteiramente da arquitetura da aplicação, e não do sistema operacional.

5.4 Questão 4 — Concordância entre psutil e a ferramenta nativa

Os resultados obtidos pelo script Python (psutil) bateram com os resultados da ferramenta nativa (ps/Gerenciador de Tarefas) em todas as atividades? Alguma diferença encontrada?

Resposta: Sim, em todas as quatro atividades, conforme consolidado na Tabela 4. Nas Atividades 1.1, 1.2 e 1.3 os valores foram idênticos dígito a dígito: os três PIDs do Bloco de Notas listados pelo script coincidiram com os exibidos na aba Detalhes do Gerenciador de Tarefas; as contagens de threads retornadas pelo `psutil` reproduziram as do `Get-Process`; e a cadeia de ancestrais reconstruída pelo script repetiu os pares PID/PPID lidos no `Get-CimInstance Win32_Process`.

Uma única diferença foi encontrada, na Atividade 1.4, e ela é de **unidade de medida, não de conteúdo**: o `monitor_cpu.py` indicava $\sim 100\%$, enquanto o Gerenciador de Tarefas mostrava $\sim 8\%$ para o mesmo processo no mesmo instante. A explicação é a base de normalização — o `cpu_percent()` do `psutil` toma um núcleo lógico como referência, ao passo que o Gerenciador expressa o consumo em relação à capacidade total da máquina. Como o processador possui 12 núcleos lógicos, $100\% \div 12 \approx 8.3\%$, e as duas leituras descrevem exatamente o mesmo fenômeno. A lição prática é que comparar ferramentas exige verificar antes qual é a base de cada percentual.

5.5 Questão 5 — Ferramentas utilizadas e obstáculos

Qual ferramenta você usou (linha de comando nativa, gráfica, ou Python) e por quê? Descreva um obstáculo prático encontrado na instalação/uso.

Resposta: Foram usadas as **três** categorias, de forma complementar: a ferramenta **gráfica** (Gerenciador de Tarefas) para a leitura rápida de PIDs e para a visualização do gráfico de CPU; a **linha de comando nativa** (PowerShell, com `Get-Process` e `Get-CimInstance Win32_Process`) para obter listas de threads e pares PID/PPID em formato tabular; e o **Python com psutil** como referência principal, por duas razões: o mesmo código roda sem alteração em Windows e Linux, e a saída fica registrada em texto, o que facilita documentar os resultados neste relatório.

Obstáculo prático encontrado: a instalação inicial do `psutil`. O comando `pip` não era reconhecido no terminal porque o Python havia sido instalado sem marcar a opção *Add python.exe to PATH*; foi necessário corrigir a variável de ambiente `PATH` antes que `pip install psutil` funcionasse. Um segundo ponto de atenção foi a Atividade 1.4, que exige **dois terminais abertos simultaneamente** — no primeiro o gerador de carga, no segundo o monitor —, já que executar os dois em sequência no mesmo terminal não permite observar o consumo enquanto ele ocorre.

5.6 Questão 6 — Diferença entre processo e thread

Com base na experiência prática, escreva com suas próprias palavras a diferença entre processo e thread.

Resposta: Com base no que foi observado nas quatro atividades, um **processo** é uma **unidade de isolamento**: é o programa efetivamente carregado na memória, com PID próprio, espaço de endereçamento próprio e recursos próprios. Foi o que se viu na Atividade 1.1, em que três janelas do mesmo executável se comportaram como três mundos separados, sem enxergar a memória umas das outras.

Uma **thread** é uma **unidade de execução dentro de um processo**: é um fluxo de instruções que o sistema operacional agenda para rodar. As threads de um mesmo processo compartilham a memória desse processo, mas cada uma mantém seus próprios registradores e sua própria pilha. Foi o que se viu na Atividade 1.2, em que um único Chrome, com um único PID, abrigava 38 fluxos de execução simultâneos.

Em resumo: **processos separam, threads compartilham**. Usam-se múltiplos processos quando se quer isolamento e segurança (o travamento de um não afeta os outros); usam-se múltiplas threads quando se quer eficiência e troca barata de dados dentro de uma mesma aplicação. O preço de cada escolha é o simétrico do seu benefício: comunicação entre processos é mais custosa, e compartilhamento entre threads exige sincronização cuidadosa.

5.7 Considerações finais

Os experimentos comprovaram que ferramentas de observação do SO — tanto nativas quanto via APIs como o `psutil` — permitem validar e aprofundar o entendimento teórico de conceitos de sistemas operacionais. A discrepância entre a expectativa teórica (múltiplas threads no processo Python) e a prática (apenas 1 thread observada) levou a uma investigação mais profunda sobre o GIL, demonstrando que a observação empírica é essencial para complementar o conhecimento teórico.

6 REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Tanenbaum, A. S., *"Modern Operating Systems"*, 4^a edição, Pearson, 2015.
- [2] Silberschatz, A., Galvin, P. B., Gagne, G., *"Operating System Concepts"*, 10^a edição, Wiley, 2018.
- [3] Documentação oficial psutil, *"psutil documentation"*, disponível em: <https://psutil.readthedocs.io/> acesso em agosto de 2026.
- [4] Microsoft Docs, *"About Processes and Threads"*, disponível em: <https://docs.microsoft.com/windows> acesso em agosto de 2026.